a friendly textbook · January 2026 exam

ServiceNow CSA

The Easy Book

Every exam topic explained in plain English, with pictures in words, tiny examples, and “why should I care?” notes. If you are new to ServiceNow, start on page 1 and do not skip the stories.

60	90	6	0
questions	minutes	domains	half marks

How to read this book

Each chapter has four layers: a **story** (so it sticks), a **simple meaning**, a **how it looks in ServiceNow**, and an **exam trap**. When you finish a chapter, try the 2-minute practice on a free Personal Developer Instance (PDI) from developer.servicenow.com.

Study order if you are short on time: **Data & security** → **Catalog & Knowledge** → **Lists & forms** → **Scripts & update sets** → **Users & plugins** → **Navigation**.

Not affiliated with ServiceNow. Original teaching notes based on the public CSA blueprint.

CONTENTS

What is inside

1. What ServiceNow actually is
2. Your instance — the house you live in
3. Moving around: Next Experience
4. Plugins, branding, and “make it ours”
5. Users, groups, and roles (the permission family)
6. Lists, filters, and tags
7. Forms, views, and the famous save buttons
8. Tasks, assignment, SLAs, and Visual Task Boards
9. Reports, dashboards, and notifications
10. Knowledge Management
11. Service Catalog — REQ, RITM, SCTASK
12. Workflow Studio and Virtual Agent
13. Tables, fields, and relationships
14. Access Control Lists (the lock on every door)
15. Importing data without making a mess
16. CMDB and CSDM — the map of the company’s IT
17. Security Center and who is responsible for what
18. UI Policy, Data Policy, Business Rules, and scripts
19. Update sets — moving your work to production
20. Putting it all together + exam method
21. Practice questions explained like a teacher

CHAPTER 1 · DOMAIN 1 · 7%

What ServiceNow actually is

If someone asks “what is ServiceNow?” and you say “a ticketing tool”, you are only 10% right. CSA wants the other 90%.

The story

Imagine a big company. People email IT: “laptop broken”, “need software”, “wifi is dead”. Managers cannot see who is overloaded. Nobody knows which server supports the payroll website. HR has a different system. Customer support has another.

ServiceNow is one **shared work platform in the cloud** where those processes can live together: incidents, requests, knowledge articles, approvals, and a map of the company’s technology. You do not install a giant server in your office for the core product. ServiceNow hosts it. You **configure** it.

Simple meaning

ServiceNow is a cloud **application Platform-as-a-Service (aPaaS)**.

- **Platform** = a base that many apps sit on (like Android is a platform for many apps).
- **as a Service** = they run the computers; you log in with a browser.
- The same platform runs IT work, employee work (HR), customer work, and custom apps you build.

CSA does not ask you to be an HR expert or a security-operations expert. It asks you to know that lists, forms, tables, roles, flows, and CMDB are the same tools everywhere.

Multi-instance (this word is loved by the exam)

Some cloud products put every customer in **one giant shared database** (multi-tenant). ServiceNow is different. Each customer gets their own **instance**: their own application and their own database.

Think of an apartment building vs a hostel dorm. Multi-tenant is a dorm: many people, one big room. Multi-instance is apartments: same building company, but your furniture and your mess stay in **your** flat.

Remember this

ServiceNow architecture = **multi-instance**, not a shared multi-tenant database.

A company usually has at least two instances:

- **Production** — real users, real data. Do not experiment here.
- **Sub-production** — development / test / UAT. This is where you try things.

A **clone** copies production onto a test instance so testers see realistic data. Cloning overwrites the target instance. It is like replacing all the furniture in the test apartment with a copy of production’s furniture.

Now Support (HI) vs working inside the instance

Day to day, you click around inside ServiceNow: create users, design forms, build catalog items.

Some jobs need ServiceNow themselves: certain upgrades, some plugins, official support cases, sometimes clones. That portal is **Now Support**, historically called **HI** (Hosted Instance).

> **Why this matters**
> If a question says “where do you request an instance upgrade?”, think Now Support / HI, not the Incident table.

Personal Developer Instance (PDI)

A PDI is a free practice instance from developer.servicenow.com. It has demo people (like Abel Tuter) and demo incidents. Break it. Reset it. It is a sandbox, not a customer production system.

Next: the instance is a house. Let’s walk through the front door.

CHAPTER 2 · DOMAIN 1

Your instance — the house you live in

An instance has a web address like `https://dev12345.service-now.com`. Dev, test, and prod have **different URLs**. If you build a catalog item in production “just this once”, you will eventually hurt real users.

What is stored in the instance?

- Applications and modules (the menu)
- Tables and records (the data)
- Users, groups, roles
- System properties (switches)
- Plugins (feature packs)
- Your customizations

Logging in — proving who you are

This is **authentication** (who are you?), which is not the same as **authorization** (what may you do?). Authorization is roles and ACLs, later in the book.

Method	In simple words
Local database	Password stored in ServiceNow on the user record
LDAP / Active Directory	Company directory checks the password; often needs a MID Server
SSO (SAML and friends)	You log in at Okta / Azure AD / Google, then land in ServiceNow
MFA	Password plus a second check (app code, SMS, ...)
Smart card (CAC / PIV)	A physical certificate card, common in government

CSA will not ask you to paste SAML XML. It may ask which method is in use, or why a local admin account still matters if SSO breaks (break-glass).

The user menu — two magic buttons

Impersonate User = wear someone else's badge for testing. You do not need their password. Always unimpersonate before you keep doing admin work.

Elevate role = turn on `security_admin` for **this session only**. Needed to create or edit Access Control rules when High Security is on. It is like asking the bank vault for a time-limited key.

> **Easy trap**
>
> Impersonate and Elevate are not the same. Impersonate = test as them. Elevate = extra power on YOUR session.

Personalization vs customization

This pair appears forever.

	Personalization	Customization
Who	Only you	Everyone on the instance
Examples	Your favorite modules, your list columns, your language	Company logo, a Business Rule, a catalog item
Update set	No	Usually yes

The **gear** on a list is personalization (often a green dot). **Configure → List Layout** changes the view for everybody who uses that view.

Next: where those buttons sit on the screen.

Moving around: Next Experience

Next Experience is the modern native screen. Mentally divide it into three parts, like a newspaper:

BANNER (top) + NAVIGATOR (left) + CONTENT FRAME (the big work area)

Banner

- **Logo** — click it to go home
- **Global search** — search many tables at once
- **Help**, notifications, **user menu**

Application navigator

- **All** — full menu
- **Favorites** — star, or drag a filter breadcrumb, or drag a record
- **History** — recently opened things (about the last 30). This is not an audit log and not CMDB history.
- **Filter box** — type to search the menu. Magic shortcut: `incident.list` opens the Incident list. `cache.do` (admin) flushes cache when a property change seems "stuck".

Application = a menu group (Incident). **Module** = an item under it (Create New, Open, All).

Training writes paths like: All → Incident → Create New.

Other doors into the same house

Not everyone uses the admin UI.

Door	Who
Next Experience lists/forms	Fulfillers and admins
Service Portal / Employee Center	Employees ordering things and reading articles
Now Mobile (iOS and Android)	Same, on a phone
Workspace (UI Builder)	Agents working cases all day
Virtual Agent	Chat: Teams, Slack, web, mobile

Remember this

The official sample: the application available to **all users** is **Self-Service**. Incident and Change need roles.

Additional comments on a ticket are visible to the caller. **Work notes** are for fulfillers. Mixing these up sends internal gossip to the customer.

Domain 1 is small. If you can sketch the three UI regions and explain impersonate vs elevate, you are fine.

CHAPTER 4 · DOMAIN 2 · 10%

Plugins, branding, and “make it ours”

Domain 2 is decorating the house and turning on extra rooms.

Plugins

A plugin is a bundle ServiceNow can switch on: extra tables, menus, and logic. Path: All → System Definition → Plugins.

- Many plugins an admin can activate.
- Some paid / store items need entitlement or Now Support.
- Some cannot be cleanly turned off. Always try in sub-production first.
- After activation you may see **Guided Setup** (empty circle = not started).

Demo data is often a **separate** plugin. PDIs already have demo data. Real customer prod usually should not load demo incidents of “Abel’s broken laptop”.

Studio is the workshop for building a scoped app. Custom tables in global scope often start with `u_`. Scoped apps use `x_company_...`. **Delegated Development** lets someone build in Studio without full admin.

Branding and properties

Logo, browser tab title, header colors, theme = **customization** for the whole company. Put them in an update set so they travel DEV → TEST → PROD.

System properties (`sys_properties.list`) are key-value switches. Example idea: deprecate homepages so people use **dashboards** instead. After a stubborn property change, try a new session or `cache.do`.

Multiple languages: activate localization plugins. Each user still picks **their** language in settings (personalization).

Homepages vs dashboards

Homepages are the old landing page. Dashboards are the current, shareable, role-based place for charts. If the exam asks how to stop people creating homepages, the answer is a **system property**, not deleting Reports.

Next: the most important admin skill — who can do what.

CHAPTER 5 · DOMAIN 2 (ALSO USED IN DOMAIN 5)

Users, groups, and roles

If you remember only one sentence from this chapter: put roles on groups, put people in groups.

The school analogy

A **user** is a student (or teacher). Table `sys_user`.

A **group** is a class: “Service Desk”, “Network Team”. Table `sys_user_group`. Groups can contain sub-groups. A person can be in many groups.

A **role** is a hall pass: “you may enter the Incident room”. Table `sys_user_role`. Roles can contain other roles, so one hall pass can include smaller passes.

ROLE (permission) → assigned to GROUP → GROUP contains USERS

Why this matters

When someone joins or leaves, you add/remove them from groups. You do not hunt through 40 roles on their user record.

Users — practical rules

- Do **not delete** people who created tickets if you can avoid it. Set `active = false` so history still makes sense.
- Locked out = too many failed logins. Admins can clear it.
- Prefer not to attach roles on the user related list. Use groups.

Roles you will see on the exam

Role	Plain meaning
`admin`	Almost everything. Still elevate for some ACL work.
`itil`	Fulfiller: incidents, problems, changes, catalog tasks.
`snc_internal` / `snc_external`	Logged-in employee vs outside customer, in the explicit-role model.
`approver_user`	Can approve without being a full technician.
`catalog_admin` / `knowledge_admin`	Own the catalog or knowledge bases.
`import_admin`	Import Sets.
`security_admin`	ACL maintenance — elevated, session only.
`flow_designer`	Build flows.
`report_publisher`	Publish reports more widely.
`personalize_form` / `personalize_list`	Change layouts for a view.

If someone "cannot see Incident"

There are three different doors:

1. Do they have a **role on the application menu**?
2. Do they have a **role on the module** (the menu item)?
3. Do they pass the **table ACL** (security on the data itself)?

A user can fail #1 (no menu) but still open a record if they paste a URL — or the opposite. Always impersonate to test.

Foundation data (company, department, location) should be set up early. CSDM calls this Foundation. It is usually **not** a CI in the CMDB.

Record numbers like INC0000042 are configured in **Number Maintenance** (prefix, digits). Not a script.

You now speak "admin". Next: the screens fulfillers live in all day.

CHAPTER 6 · DOMAIN 3 · 20%

Lists, filters, and tags

A **list** is a spreadsheet of records. Each **row** is one record. Each **column** is one field.

Parts of a list

- Title bar — name, count, search, view
- Breadcrumbs — the current filter, clickable like a trail of cookies
- Column headers — sort, filter, charts, export, configure
- Rows — the data. Double-click a cell to edit in place if list editing is allowed.

Right-click menus are a CSA favourite: list title, column header, a cell, a row, and the form header are **different menus**.

Filters in one sentence

A condition is always Field + Operator + Value, for example Active is true, Priority is 1. The breadcrumb shows that sentence. Click an earlier crumb to drop later conditions.

You can also click a cell → Show Matching / Filter Out, or type a natural-language query if the instance allows it. Advanced people use encoded queries like `active=true^priority=1`.

Save a filter as Me / Group / Everyone. Drag the breadcrumb to Favorites to pin it.

> **Easy trap**
>
> Gear (personalize columns) is only for you. Configure → List Layout is for everyone on that view. The gear shows a green dot when you personalized.

Tags are sticky labels you put on records for ad-hoc grouping ("release week"). They are not categories (a real field) and not user criteria (who can see a catalog item).

Wildcard * often means "contains" in list search. If the question says "search anywhere in the field", think wildcard.

Lists find work. Forms are where you do the work.

CHAPTER 7 · DOMAIN 3

Forms, views, and the famous save buttons

A form is one record opened up. Header buttons, field sections, related lists at the bottom, activity stream (comments and work notes), maybe a process-flow formatter.

A **reference field** looks like a name but stores a `sys_id` (a 32-character unique ID). The lookup icon opens a popup list. **Dot-walking** follows the arrow: Incident → Caller → Department → Name.

Views

A view is a named layout of the same table: Default for fulfillers, Self Service for callers, Workspace for agents. Changing Default does **not** change Self Service. If a user "cannot see a field" but can see the record, first ask: **wrong view?**

Form Layout (slushbucket) and **Form Designer** (drag-and-drop canvas) write the same layout records. In designer: handle = move, remove = off the form (field still exists in the dictionary), edit = field settings. Adding a brand-new field is a data-model change.

Related lists show records that point at this one, or many-to-many links (Affected CIs on an incident).

Memorize this table cold

Button	New record?	Form closes?
Submit	Yes — creating	Yes
Update	No — saving existing	Yes
Save	No (or first insert of a new form)	**No — stays open**
Insert	**Yes** — clones current values to a new sys_id	Yes
Insert and Stay	Yes — new sys_id	**No — stays open**

Easy trap

Insert is not Submit. Insert copies an **existing** record into a new row. Update is not Save. Update closes; Save stays.

Templates pre-fill fields for speed. They are not security. ACLs still apply.

If a field must always be mandatory for everyone, use the dictionary or a Data Policy. If it is mandatory only when Priority is 1 **on the form**, use a UI Policy. If the question is about hiding data from people who should not see it, that is an **ACL**, not a UI Policy.

A **dictionary override** lets a child table (Incident) change a parent field's behaviour (from Task) without changing Problem and Change.

Next: almost every process ticket is a kind of Task.

CHAPTER 8 · DOMAIN 3

Tasks, assignment, SLAs, Visual Task Boards

`task` is a **parent table**. Incident, Problem, Change, catalog requests and catalog tasks all **extend** it. They share number, short description, state, assignment group, assigned to, priority, work notes, comments, SLAs.

If you add a field to Task, children inherit it. That is powerful and dangerous. Prefer adding fields on the child unless you truly want it everywhere.

My Work vs My Groups Work

Module	Meaning
My Work	Active tasks assigned **to me**
My Groups Work	Assigned to **one of my groups**, usually not yet to a person

> **Remember this**
> Official sample wording is **My Groups Work**. 'My Teams Work' is a fake friend.

How a ticket gets an owner

- Someone assigns it by hand
- **Assignment rules** (System Policy → Rules → Assignment) match conditions → group and/or user
- Older Incident lookup rules
- Predictive Intelligence (machine learning suggestions)
- A flow or Business Rule

Best practice in training: assign a **group** from category/subcategory; only assign a person when you are sure.

Clocks

Clock	Between whom
SLA	You and the customer
OLA	Internal teams
UC (underpinning contract)	You and a supplier
Inactivity monitor	Nobody touched the ticket — not a contract

An SLA on a task **starts**, can **pause** (waiting on customer), and **stops** (resolved). Each attached clock is a row on `task_sla`.

Visual Task Boards (VTBs)

Kanban cards. Three types:

- **Guided** — lanes come from a field (State). Dragging a card **updates that field**.
- **Flexible** — mostly a field, plus extra lanes you add.
- **Freeform** — you invent lanes. Do not assume it changes Incident State.

The board **owner** can change lanes and members. Members work the cards. VTBs help collaboration; they do not replace security.

People also need charts and emails. That is the next chapter.

CHAPTER 9 · DOMAIN 3

Reports, dashboards, and notifications

Reports

Open Reports → View / Run, or right-click a list column → bar/pie chart (fastest demo).

Report Designer tabs, in order:

Data → Type → Configure → Style

- **Data** — name, table or data source
- **Type** — bar, pie, line, score, ...
- **Configure** — group by, aggregation (count, average, ...), filters
- **Style** — colours and titles

By default only the creator sees the report. The recommended way to give it to colleagues is **Share** with users, groups, or roles. Publish makes a wider URL (be careful). Schedule emails it. Add to dashboard places it as a widget. Homepages are legacy; dashboards are current.

Platform Analytics is the newer name for visualisations and KPI signals. CSA does not need Performance Analytics admin depth.

Notifications

Path: System Notification → Email → Notifications. Usually a **system event** fires (incident inserted, a flow fires an event), then three questions:

Tab	Question
When to send	Which table, insert/update, conditions, which event?
Who will receive	User, group, Assigned to, Caller, ...
What will it contain	Subject, body, SMS, calendar invite

Users can subscribe/unsubscribe in System Settings (personalization). Admins can force some legal mails. Channels: email, SMS, push, meeting invite, digest.

The instance can also **receive** mail. An inbound email action can create an incident from a new message or update a ticket from a reply (watermark / number in the subject).

Easy trap

If mail did not arrive: did the event fire? Did conditions match? Is the user active with a good email? Is the instance allowed to send mail (often off in sub-prod)? Did they unsubscribe?

CHAPTER 10 · DOMAIN 4 · 20%

Knowledge Management

Knowledge is the company's "how do I...?" library. The goal is to answer the question **without opening a ticket**.

Record	Table	Job
Knowledge base	`kb_knowledge_base`	A library with an owner
Category	`kb_category`	Folders inside the library
Article	`kb_knowledge`	The actual page
User criteria	`user_criteria`	Who may read or write

Can Read and **Can Contribute** are different keys. A technician might write IT articles but only read HR articles.

User criteria can match users, groups, roles, company, location, or a script. If an article is invisible, check published state **and** user criteria. Impersonate an employee with no itil role.

Life of an article

Draft → Review → Published (searchable) → Retired

Users search **published** articles. Retired ones stay for history. Publishing may create a new version.

Need	Use
How do I reset VPN?	Knowledge article
I want a laptop	Catalog item
VPN is down for me right now	Incident (often a record producer)

A record producer can still show knowledge suggestions before submit (deflection).

CHAPTER 11 · DOMAIN 4

Service Catalog — shopping for work

The catalog is the ordering system. Hardware, software, access, and often “report an issue” forms live here.

The family tree (draw this in the exam)

When someone orders a normal **catalog item** (not a record producer):

REQ (sc_request) — the shopping cart checkout
→ RITM (sc_req_item) — one per item in the cart
→ SCTASK (sc_task) — work for fulfiller groups

One checkout = one REQ. Two laptops in the cart = two RITMs under that REQ. Each RITM can have several tasks (hardware team, software team).

Easy trap

A **record producer** skips this tree. It creates a row on a target table (often Incident). No REQ unless you build that yourself.

Item types

Type	What it does
Catalog Item	Normal order → REQ / RITM / SCTASK
Record Producer	Creates a record on another table
Order Guide	One form that orders **several** items (new joiner: laptop + phone + access)
Content Item	A link or knowledge — not an order

Remember this

Official sample: Multiple Choice, Single Line Text, Select Box are **variable types** — questions on the item — not order guides.

A **variable set** is a reusable bundle of questions (Requested for + Location). Change once, many items update.

After submit, a **Flow** should approve, create catalog tasks, and close the RITM. If you forget the flow, you get a request that sits there looking sad.

Who sees the item? Active flags + **user criteria** on catalog, category, and item. Always test as a user with no itil role.

CHAPTER 12 · DOMAIN 4

Workflow Studio and Virtual Agent

On the January 2026 blueprint the umbrella name is **Workflow Studio**.

Tool	Job
Flow Designer	When X happens, do Y. Can wait.
Process Automation Designer / playbooks	Guided stages for agents (qualify → investigate → resolve)
Workflow Editor	Old drawing-board workflows. Maintain old ones; do not start new work here if Flow can do it.

A flow = **trigger** + **actions** (if, for each, wait, create record, ask approval, create catalog task, send email). A **spoke** is a pack of actions (ServiceNow Core, Slack, Teams, REST). A **subflow** is a reusable mini-flow with no trigger of its own — like a function.

Triggers: record changes, a date, Service Catalog, inbound email, SLA task.

Roles: `flow_designer` builds, `action_designer` builds actions, `flow_operator` watches executions. If a catalog item “does nothing”, open today’s flow executions.

> **Remember this**
>
> A Flow can **wait** two days for a manager to approve. A Business Rule must not wait — it runs in the same database save.

Virtual Agent

The chat bot for employees. It searches **Knowledge**, can start **catalog** orders, checks ticket status, and can hand off to a live person. It appears on the portal, Now Mobile, Microsoft Teams, Slack, and web chat.

You will not build a full language model in the CSA exam. You must know VA is a self-service channel that sits on knowledge + catalog + records.

You can now describe the employee journey. Next is the biggest domain: data and locks.

CHAPTER 13 · DOMAIN 5 · 30% STARTS HERE

Tables, fields, and relationships

Everything in ServiceNow is a row in a table. Incidents, users, even Business Rules. Once you see that, the platform gets quieter.

Kinds of tables

Kind	Example
Base (others extend it)	`task`, `cmdb`
Child / extended	`incident` **is a** task
Custom global	`u_loaner`
Scoped app	`x_acme_loan_loaner`

Label = Incident. Name = `incident`. The exam loves database names: `sys_user`, `task_sla`, `cmdb_ci`.

Every record has a **sys_id**: 32 characters, globally unique. INC0001234 is a friendly number from Number Maintenance. The real primary key is sys_id.

Three ways tables connect

1. **Extension** — "is a". Incident is a Task. Child inherits fields. `sys_class_name` remembers the real class (important in CMDB).
2. **One-to-many** — a **reference field** on the many side. Many incidents point at one caller. On the user form, a related list can show that user's incidents.
3. **Many-to-many** — a joining table with two references, for example incidents ↔ CIs (Affected CIs).

The **Schema Map** (from Tables & Columns) draws these links. Use it when a question says "visualize table relationships".

The **Dictionary** (`sys_dictionary`) is the ID card of every field: type, length, mandatory, default, reference table. A **dictionary override** changes a parent field only on a child table.

Dot-walking follows references in filters, notifications, and scripts: `caller_id.department.name`. In the condition builder a green plus expands the reference.

Application **scope** is a fence around an app's files so random global scripts do not wander in. User security is still ACLs, next chapter.

CHAPTER 14 · DOMAIN 5 · THE LOCK

Access Control Lists

An ACL is a rule that answers: **who may do this operation on this object?** Operations: create, read, write (update), delete, plus extras like execute.

Why this matters

UI Policy can hide a field on a form. A clever person or an import can still touch the data. ACL is the real lock.

The three names

ACL name	What it locks
`incident` or `incident.None`	The whole row — may I see this ticket at all?
`incident.*`	Fields in general
`incident.work_notes`	That one field

You must pass the **table/row** ACL first. If that fails, field ACLs cannot save you. If table read passes but work_notes read fails, the incident opens and that field is blank.

AND vs OR — write this on scrap paper

Inside **one** ACL record there are up to three parts: roles, a condition, a script. All configured parts must pass. That is AND. Empty parts are skipped.

If **several** ACL records match the same object and operation, passing **any one** of them typically grants access. That is OR.

More specific names win: `table.field` beats `table.*`.

On modern High Security instances, if nothing grants access, the answer is **deny**.

1. Elevate to `security_admin` (session only).
2. Open System Security → Access Control, or the table's Access Controls related list.
3. Create the rule, then **impersonate** a normal user and prove it.

Easy trap

admin override on an ACL can let admins bypass it. Clearing that checkbox is how you lock admins out of a sensitive field — rare, but know the checkbox exists.

Authentication = who are you (password/SSO). Authorization = what may you do (roles + ACLs). Query Business Rules can also hide rows silently — another thing to check when "the list is empty".

Importing data without making a mess

Do not type 5,000 users by hand. Use **Import Sets**.

File or data source → staging table (parking lot) → Transform Map (the translator) → target table (sys_user, cmdb_ci, ...)

Load Data: Excel, CSV, XML, or a repeating Data Source (JDBC, LDAP, HTTP). Roles: admin or import_admin.

Remember this

Official definition: a **transform map** maps Import Set fields to fields on an **existing** table.

If column names already match, Automatic Mapping helps. If not, Mapping Assist (drag fields). Scripts can set extra fields.

Coalesce — the most important import word

Mark a unique key (email, employee number, serial number) as **coalesce**:

- Match found → **update** that person/CI
- No match → insert a new row
- Several coalesce fields → all must match (AND)
- No coalesce → every load **inserts duplicates**

Tiny example

Load users twice with coalesce on email. Second load only changes names. Load without coalesce and you get twins.

Easy trap

UI Policies do not run during import. If a field must be mandatory even from Excel, use a **Data Policy** (or dictionary mandatory).

Update sets move configuration, not 10,000 user rows. Clone copies a whole instance. Import Sets move data.

CMDB and CSDM — the map of the company's IT

CMDB = Configuration Management Database. It stores **Configuration Items (CIs)** — the things you manage — and **how they connect**.

CIs can be physical (servers, laptops, routers) or logical (a business service, an application service, a database).

> **Why this matters**
>
> When a server dies, you want to know which business service, which customers, which changes. Incidents and changes should point at CIs. That is impact analysis.

Table	Job
`cmdb` / `cmdb_ci`	CI classes
`cmdb_rel_ci`	This CI depends on / runs on / uses that CI

CI Class Manager is the modern place to see the class tree. **Dependency Views** draw a map. **Suggested Relationships** say which relationship types make sense between two classes.

How data gets in

- **Discovery** — bottom-up: find devices on the network. Needs a **MID Server** (Management, Instrumentation, and Discovery): a small service **inside your network** that talks out to the instance.
- **Service Mapping** — top-down: start from a website and walk down to the servers.
- Connectors / IRE (Identification and Reconciliation) — stop duplicate CIs.
- Manual forms and Import Sets — still exist, but identity rules matter.

CSDM in plain words

CSDM (Common Service Data Model) is not "another product you buy instead of ITSM". It is ServiceNow's **agreed vocabulary and table map** so everyone stores services the same way.

- **Foundation** — users, groups, locations, companies. Referenced everywhere. Usually not linked with CI relationship records.
- **Design** — Business Application: the idea of "Payroll App" in architecture. Not the running server.
- **Build** — software-development objects.

- **Manage technical services** — Application Service (a deployed running instance) plus infrastructure CIs Discovery finds. This is what often breaks at 3 a.m.
- **Sell / Consume** — Business Service and offerings: what the business sells or uses. The **request catalog is not a CMDB table**.
- **Manage Portfolio** — service owners looking across the above.

> **Easy trap**
> Business Application (design) is not the same as Application Service (running instance). The catalog is not stored as cmdb_rel_ci.

CHAPTER 17 · DOMAIN 5

Security Center and shared responsibility

Security Center is a dashboard about the **health of the instance's own security settings**: hardening checks, risky properties, recommendations. It is not Security Incident Response (hacking cases), and it is not a list of firewall CIs in the CMDB.

If the question is "where do admins review instance security posture in one place?", Security Center.

Shared Responsibility — the cloud split

ServiceNow runs a cloud platform. They cannot log in and design your HR ACLs for you. You cannot fly to their data centre and patch a cable.

ServiceNow typically owns	You (the customer) typically own
Physical data centres and hardware	Who gets admin / security_admin
Keeping the platform software running	Users, groups, roles, ACLs
Instance isolation (multi-instance)	What data you put in the instance
Offering Security Center and guidance	Acting on those findings
Cloud operations	MID Servers inside **your** network, integrations, public pages you published

> **Remember this**
> Who is responsible for designing ACLs that protect HR data? **The customer's admins.**
> Who is responsible for physical security of the data centre? **ServiceNow.**

CHAPTER 18 · DOMAIN 6 · 13%

UI Policy, Data Policy, Business Rules, and scripts

The exam loves “which tool?”. Pick the **simplest tool that actually enforces the need**.

Decision tree in slow motion

If you need…	Use
Mandatory / visible / read-only only on the **form**	**UI Policy** (browser, no code)
The same rule even from Import Set, API, list edit	**Data Policy** (server)
Wait for a manager for two days	**Flow**
Set a field or stop a save in the same click	**before Business Rule**
Slow extra work after save	async Business Rule or a Flow
Fancy onChange behaviour	Client Script (onLoad / onChange / onSubmit / onCellEdit)
Reusable server function / Ajax	Script Include
Stop people who should not see data	**ACL**

UI Policy

No-code. When a condition is true, make fields mandatory, visible, or read-only. Usually reverse if false. Runs in the browser. Not security.

Data Policy

Looks similar (mandatory / read-only) but runs on the **server**, so imports and web services must obey too. Can also show as a UI Policy on the form.

Business Rules

Server logic when a record is inserted, updated, deleted, or queried. Table `sys_script`.

When	In simple words
before	While saving, before it is committed. Set fields, abort, or (on query) filter rows.
after	Right after this record saved. Touch other records. Keep it light.
async	Queued later. Good for slow work.
display	When the form loads. Put notes in `g_scratchpad` for Client Scripts.

Scripts use `current` (this record) and `previous` (old values on update). Language of the platform: **JavaScript** — official sample. Not Java. Not Jelly.

Client vs server — two rooms

Browser: Client Scripts, UI Policies, `g_form`, `g_user`, `g_scratchpad`.

Server: Business Rules, Script Includes, `gs`, `GlideRecord`, `current`.

> **Easy trap**
> GlideRecord inside a Client Script makes a slow round-trip to the server. Use GlideAjax + a Script Include instead.

CHAPTER 19 · DOMAIN 6

Update sets — moving your work

An update set is a suitcase of **configuration changes**: new fields, Business Rules, form layouts, catalog items, ACLs, many properties.

It is usually **not** a suitcase of data: not last year's incidents, not 4,000 users, not CMDB servers. For data, use Import Sets (or clone a whole instance).

DEV: create a named set → make it current → do the work → mark Complete
TEST/PROD: retrieve or import XML → Preview → fix errors → Commit

> **Easy trap**
> Never develop in the Default update set and then migrate Default as your release. If you forgot, move the customer updates into a named set first. Always Preview before Commit.

Customer Updates related list shows what was captured. If something is missing, you were in the wrong set, or that object is data (not captured).

Scoped apps can also move as app versions / Git. CSA still tests classic update set preview and commit.

A committed set can sometimes be backed out. That is an emergency tyre, not a process.

CHAPTER 20

Putting it all together

One employee story

Maya's laptop is slow. She opens Employee Center or Virtual Agent.

A **knowledge** article says "restart, then order a loaner if still bad". Still bad. She orders **catalog item** "Loaner laptop". Variables ask size and reason.

ServiceNow creates REQ + RITM. A **flow** asks her manager to approve, then creates an SCTASK for the Service Desk group (assignment rule could also help). A **notification** emails the group.

The technician works My Groups Work, maybe a Guided VTB by State. They link a **CI** (the laptop in the CMDB). An **SLA** ticks. Comments go to Maya; work notes stay internal.

You, the admin, built this in DEV inside a named **update set**, previewed it in TEST, committed to PROD. You tested by **impersonating** Maya (no itil role) and a technician.

If Maya should not see HR articles, that is **user criteria**. If a contractor should not see work notes, that is an **ACL**. If justification is mandatory only when "replacement" is selected, that is a **catalog UI policy**. If justification must exist even when someone imports requests, that is a **data policy**.

Exam method

- 60 questions, 90 minutes, no half marks on "select 3".
- First pass fast. Flag. Second pass for ACL / coalesce / save buttons.
- When two answers could work, pick the most specific platform feature (UI Policy before Client Script; Data Policy if it must hold on the server; Flow if it must wait; ACL if it is access).
- Confirm the exam **release** on docs.servicenow.com.

Keep the cert alive with the annual delta exam. Official courses: Welcome to ServiceNow + ServiceNow Administration Fundamentals.

CHAPTER 21

Practice questions — explained like a teacher

These are original drills, not leaked live items. The five official blueprint samples are included first.

Official 1. Which application is available to all users?

Self-Service. People with no fulfiller role still get a place to read knowledge and order from the catalog. Incident/Change need roles.

Official 2. Tasks for my group, not yet for a person?

My Groups Work. My Work = assigned to me.

Official 3. Transform map?

Maps Import Set (staging) fields to an existing target table.

Official 4. Multiple Choice, Single Line Text, Select Box?

Variable types on a catalog item.

Official 5. Scripting language?

JavaScript.

More drills (answer then reason)

1. **Multi-instance** — each customer has their own DB, not one shared dormitory database.
2. **Impersonate** to test as an employee. Elevate is for security_admin on your session.
3. Roles on **groups**, users in groups.
4. Gear = personalization. Configure layout = customization for the view.
5. Insert and Stay = new sys_id, form stays open.
6. Filter = field + operator + value.
7. Guided VTB lanes follow a field; dragging updates it. Freeform does not reliably change Incident State.
8. Share is the recommended way to give someone a report.
9. Notifications: When / Who / What, usually from an event.
10. Two catalog items in one cart = one REQ, two RITMs.
11. Record producer writes a target table; no REQ chain by default.
12. Flows wait; Business Rules do not.
13. sys_id is 32 characters. INC numbers are labels.
14. Incident extends Task = parent-child extension, not many-to-many.

15. One ACL's parts = AND. Several ACLs at the same spot = typically OR. Table then field.
16. No ACL + default deny = denied.
17. Coalesce on email = second import updates, does not twin people.
18. cmdb_ci and cmdb_rel_ci are CMDB. sc_request is catalog.
19. CSDM = common model/vocabulary. Discovery needs MID in your network.
20. Customer owns ACL design. ServiceNow owns the data centre.
21. Priority 1 makes Description mandatory on the form = UI Policy. Same rule on import = Data Policy.
22. display Business Rule fills g_scratchpad for the client.
23. Update sets = configuration. Preview then Commit. Named set, not Default.
24. g_form is client. UI Policy is not a lock. Catalog is not a CMDB table. Hidden module but URL works = module role missing, not table ACL.

You can explain the platform now — not just tick boxes. Go and click on a PDI.

End of the Easy Book. Pair with the 15-page visual PDF for last-week skim, and the markdown workbook for extra labs.